

ARP 01 | Gestão de frotas logísticas

Documentação básica do projeto de Arquiteturas de Sistemas Distribuídos

ALUNA	Anny Gabrielly Gonçalves de Oliveira
MATRÍCULA	2310019
INSTITUIÇÃO	UniEVANGÉLICA - Universidade Evangélica de Goiás

1. Objetivo

O sistema recebe solicitações de transporte com origem, destino e peso da carga. Ele registra o pedido imediatamente, aloca um veículo compatível em segundo plano e permite consultar o status por uma API REST. O contexto é predominantemente um sistema distribuído de informação, pois coordena dados, estados e integração entre serviços.

2. Componentes e comunicação

Componente	Responsabilidade / protocolo
Cliente e API Gateway	Cliente usa REST/JSON; gateway autentica com X-API-Key e encaminha pedidos.
Solicitações	Persiste estado inicial/final em banco próprio; publica eventos por outbox.
Broker	Distribui eventos para filas por assinatura; confirmações, tentativas e DLQ.
Despacho A e B	Consumem a mesma fila, reservam por TCP e sincronizam cache via TCP P2P.
Frota	Seleciona atômicamente o veículo com capacidade suficiente em banco próprio.
Notificações	Consome eventos e registra notificações em banco isolado.

3. Fluxo principal

1. POST /requests responde 202 e CREATED depois de salvar a solicitação. 2. O broker entrega request.created a Despacho e Notificações. 3. Despacho usa TCP para reservar na Frota e publica assignment.decided. 4. Solicitações muda para ASSIGNED (com vehicle_id) ou REJECTED e publica request.status_changed. 5. GET /requests/{uuid} mostra o estado atual.

4. Requisitos atendidos

RF01-RF12: criação e validação, serviços independentes, eventos, consulta, TCP interno, P2P, logs com correlation_id, tratamento de falhas com DLQ, API key e rotas documentadas. RNF01-RNF09: falha de consumidor isolada, instâncias concorrentes, descoberta configurável, consistência eventual, rastreamento, execução local, segredos fora do código, resposta não bloqueante e serviços reiniciáveis separadamente. O mapeamento detalhado está em docs/documentacao-tecnica.md.

5. Semântica de entrega e limites

A entrega de eventos é at-least-once (pelo menos uma vez): mensagens não confirmadas podem ser entregues novamente. Reservas por `request_id` e atualizações de estado são idempotentes. Em erros explícitos, há três tentativas e depois DLQ. Os registros têm `correlation_id` em todos os serviços. A sincronização P2P de cache é de melhor esforço; o banco de Frota é a fonte autoritativa. O protótipo registra notificações internas, sem enviar SMS ou e-mail.

6. Como executar e demonstrar

Instale Node.js e Python 3.12. Na pasta do projeto, execute `npm run dev` e abra `http://localhost:8100`. Registre origem, destino e peso; acompanhe o status na tela. O comando cria `.env` com uma `API_KEY` aleatória na primeira execução. A tela usa a chave automaticamente; não é necessário defini-la ou digitá-la. Use Ctrl+C para encerrar. Para testar com Docker Compose, execute `npm run docker:dev` (Docker instalado). A coleção Postman e o roteiro de evidências estão em `docs/`.

7. Entregáveis e verificação

O pacote inclui código-fonte por serviço, `Dockerfile`, `docker-compose.yml`, `README`, diagrama Mermaid, documentação técnica, coleção Postman e teste automático. O teste local verificou autenticação, validação, alocação, TCP, P2P, notificações e DLQ. A execução via Docker Compose não foi testada no ambiente de autoria, pois Docker não estava disponível.

8. Repositório público e proteção da chave

Código e documentação: github.com/annygabb/gestaodefrotas. O arquivo `.env` real é criado somente no computador do usuário e está excluído do Git e do contexto de build do Docker. O repositório não contém `.env` nem `.env.example`. A rota `/dev-config` informa a chave à interface apenas no ambiente de teste local, ligado a 127.0.0.1; não deve ser publicada na internet.